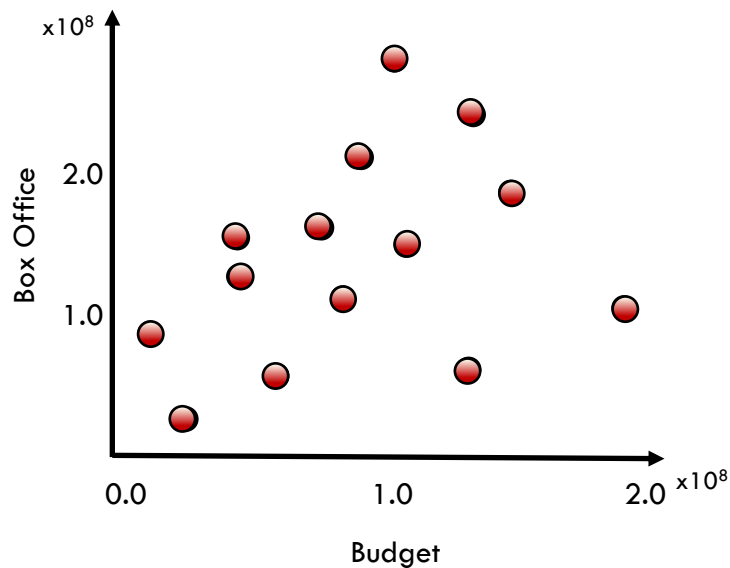


Introduction to

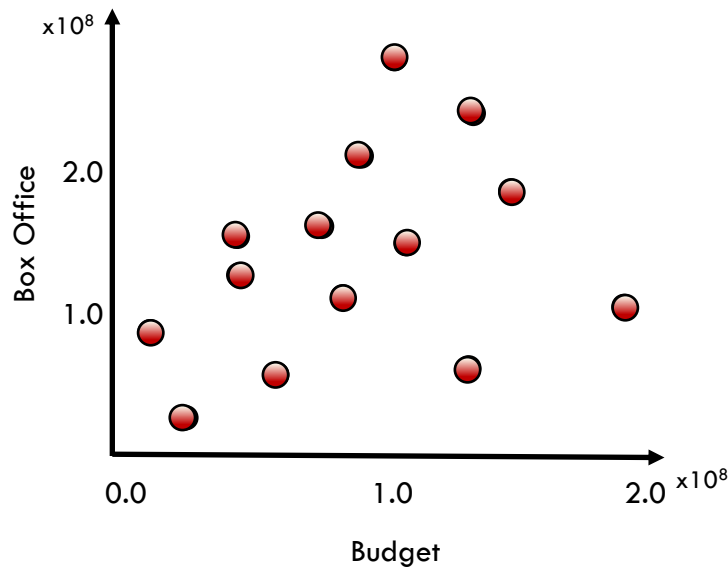
Linear Regression

Introduction to Linear Regression



$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

Introduction to Linear Regression

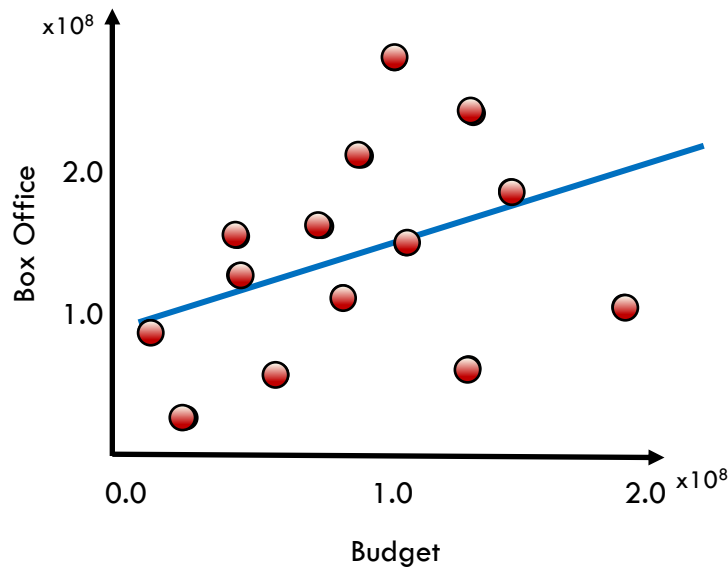


$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

Diagram illustrating the linear regression equation $y_{\beta}(x) = \beta_0 + \beta_1 x$ with annotations:

- box office revenue** points to $y_{\beta}(x)$
- coefficient 0** points to β_0
- coefficient 1** points to β_1
- movie budget** points to x

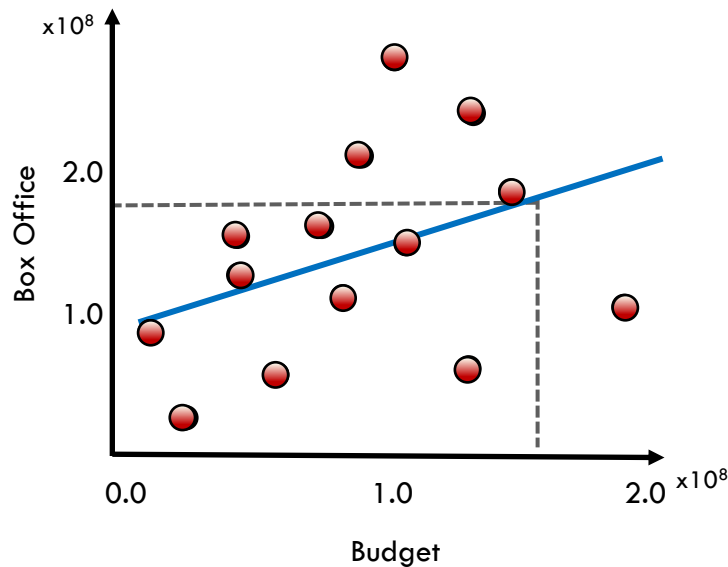
Introduction to Linear Regression



$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

$$\beta_0 = 80 \text{ million}, \beta_1 = 0.6$$

Predicting from Linear Regression

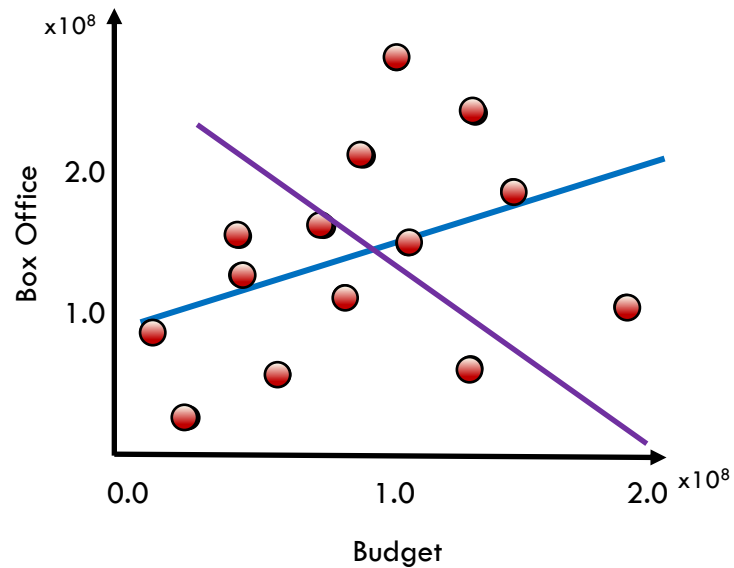


$$y_{\beta}(x) = \beta_0 + \beta_1 x$$

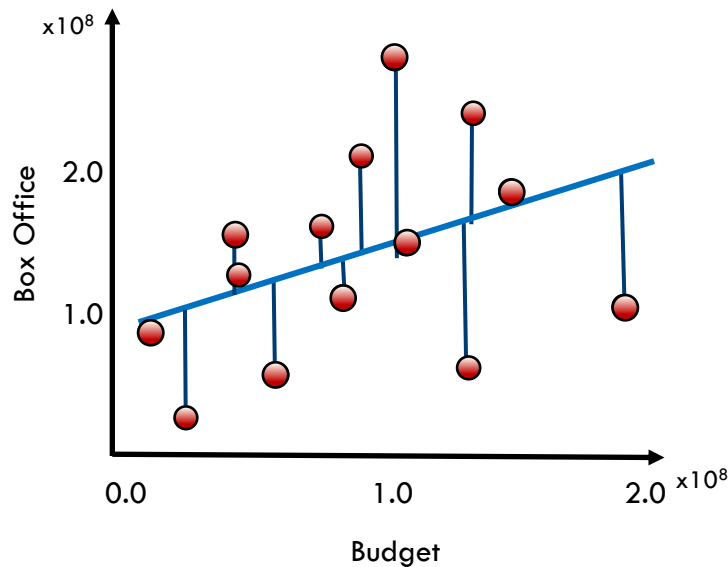
$$\beta_0 = 80 \text{ million}, \beta_1 = 0.6$$

Predict 175 Million Gross for 160 Million Budget

Which Model Fits the Best?

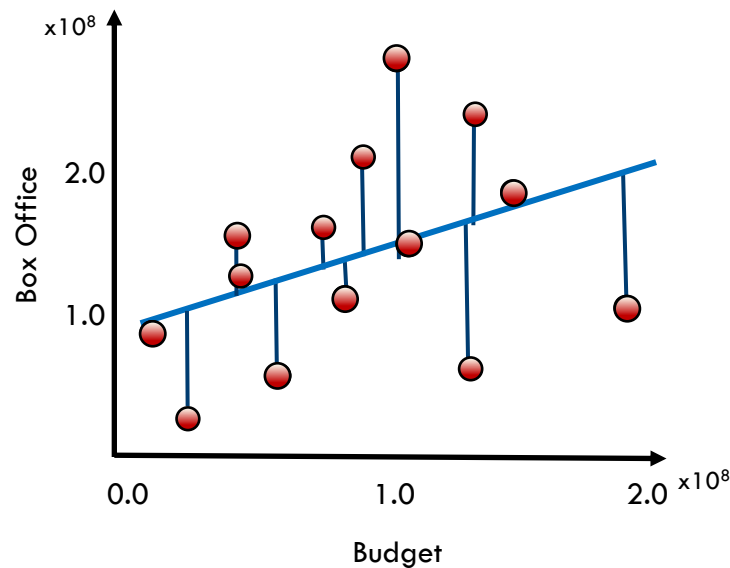


Calculating the Residuals



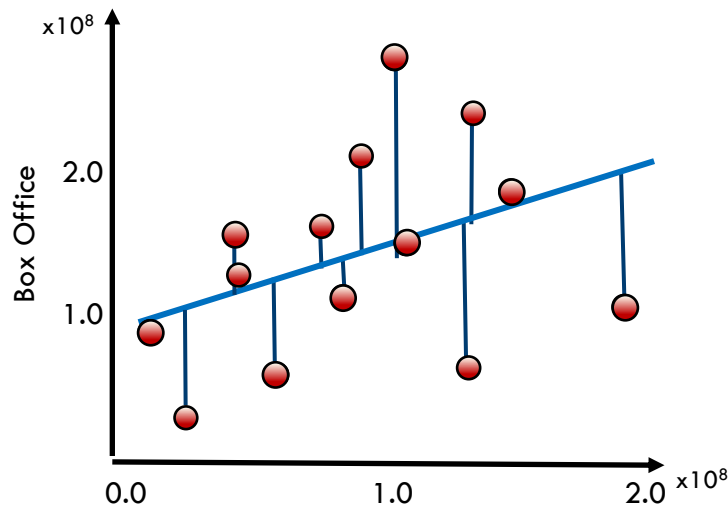
predicted value $\rightarrow y_{\beta} \left(x_{obs}^{(i)} \right) - y_{obs}^{(i)}$ $\leftarrow$ **observed value**

Calculating the Residuals



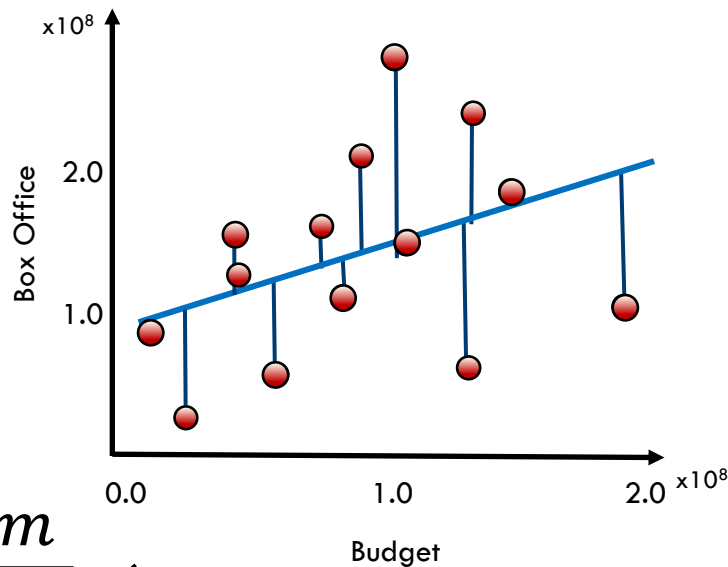
$$\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)}$$

Mean Squared Error



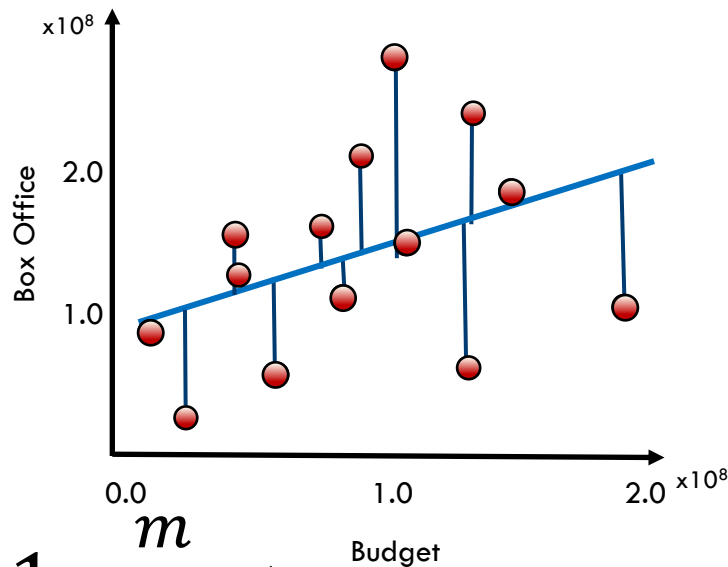
$$\frac{1}{m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Minimum Mean Squared Error



$$\min_{\beta_0, \beta_1} \frac{1}{m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Cost Function



$$J(\beta_0, \beta_1) = \frac{1}{2m} \sum_{i=1}^m \left(\left(\beta_0 + \beta_1 x_{obs}^{(i)} \right) - y_{obs}^{(i)} \right)^2$$

Modelling Best Practice

- Use cost function to fit model
- Develop multiple models
- Compare results and choose best one

Other Model Metrics

Sum of Squared Error (SSE):

$$\sum_{i=1}^m \left(y_{\beta}(x^{(i)}) - y_{obs}^{(i)} \right)^2$$

Other Measures of Error

Sum of Squared Error (SSE):
$$\sum_{i=1}^m \left(y_{\beta}(x^{(i)}) - y_{obs}^{(i)} \right)^2$$

Total Sum of Squares (TSS):
$$\sum_{i=1}^m \left(\overline{y_{obs}} - y_{obs}^{(i)} \right)^2$$

Correlation Coefficient (R^2):
$$1 - \frac{SSE}{TSS}$$

Linear Regression: The Syntax

Import the class containing the regression method

```
from sklearn.linear_model import LinearRegression
```

Create an instance of the class

```
LR = LinearRegression()
```

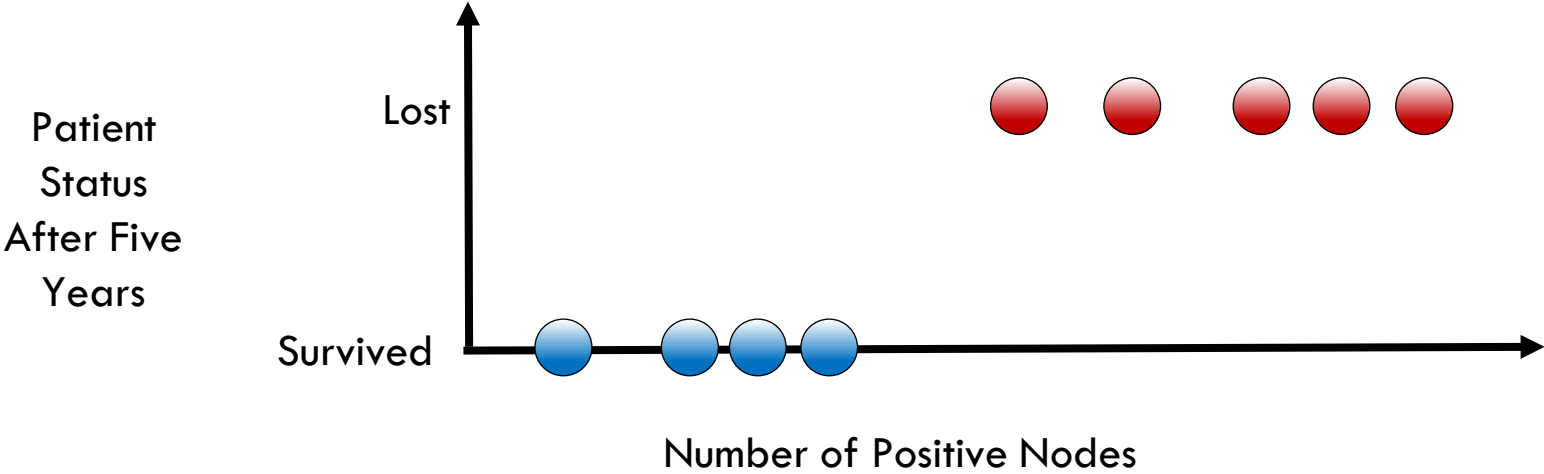
Fit the instance on the data and then predict the expected value

```
LR = LR.fit(X_train, y_train)
```

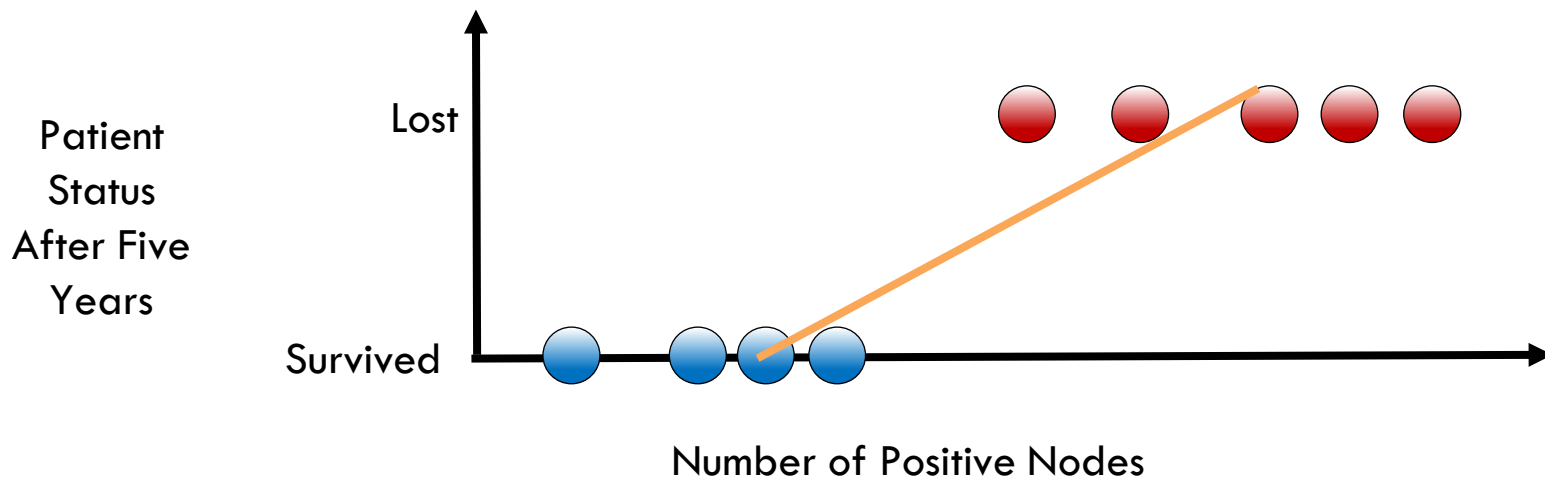
```
y_predict = LR.predict(X_test)
```

Logistic Regression

Introduction to Logistic Regression

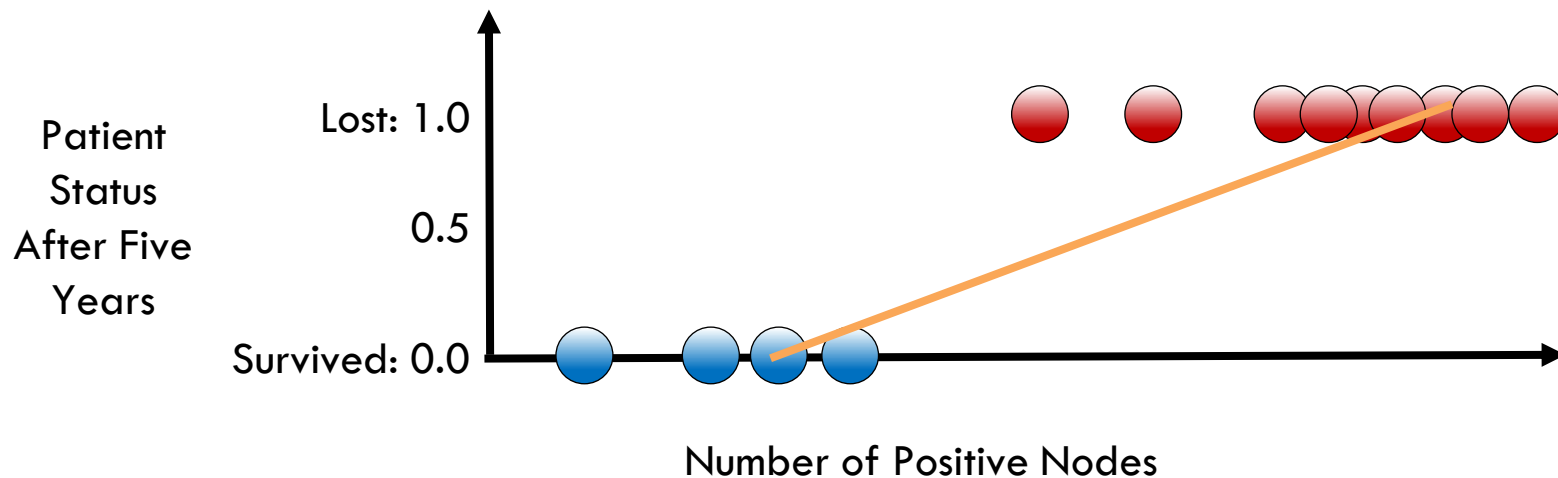


Linear Regression for Classification?



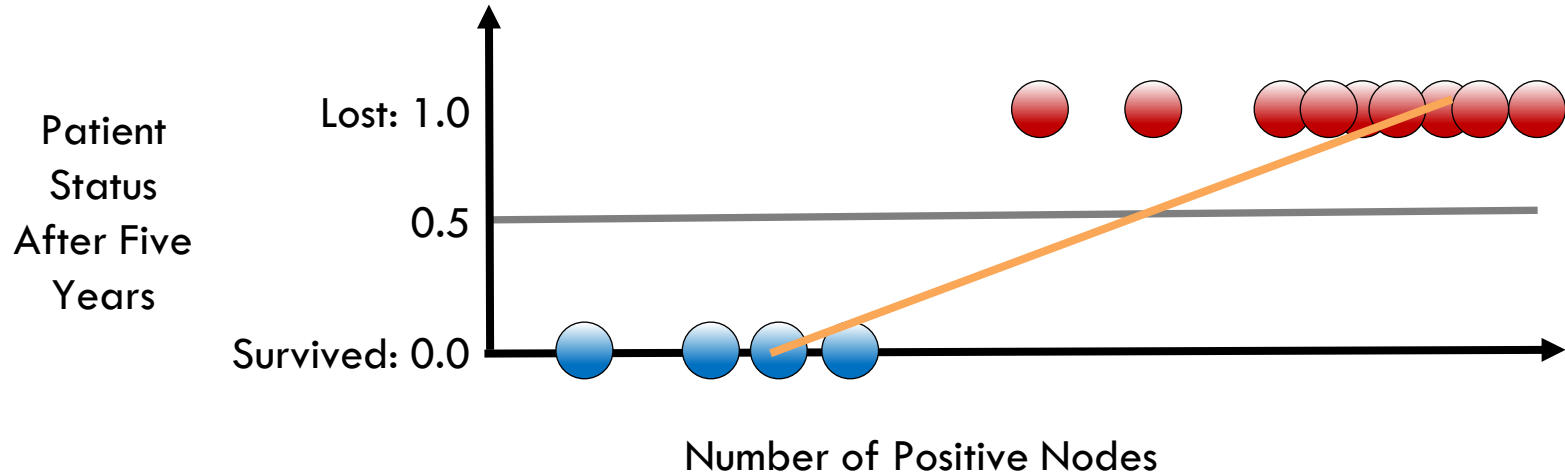
$$y_{\beta}(x) = \beta_0 + \beta_1 x + \varepsilon$$

Linear Regression for Classification?



$$y_{\beta}(x) = \beta_0 + \beta_1 x + \varepsilon$$

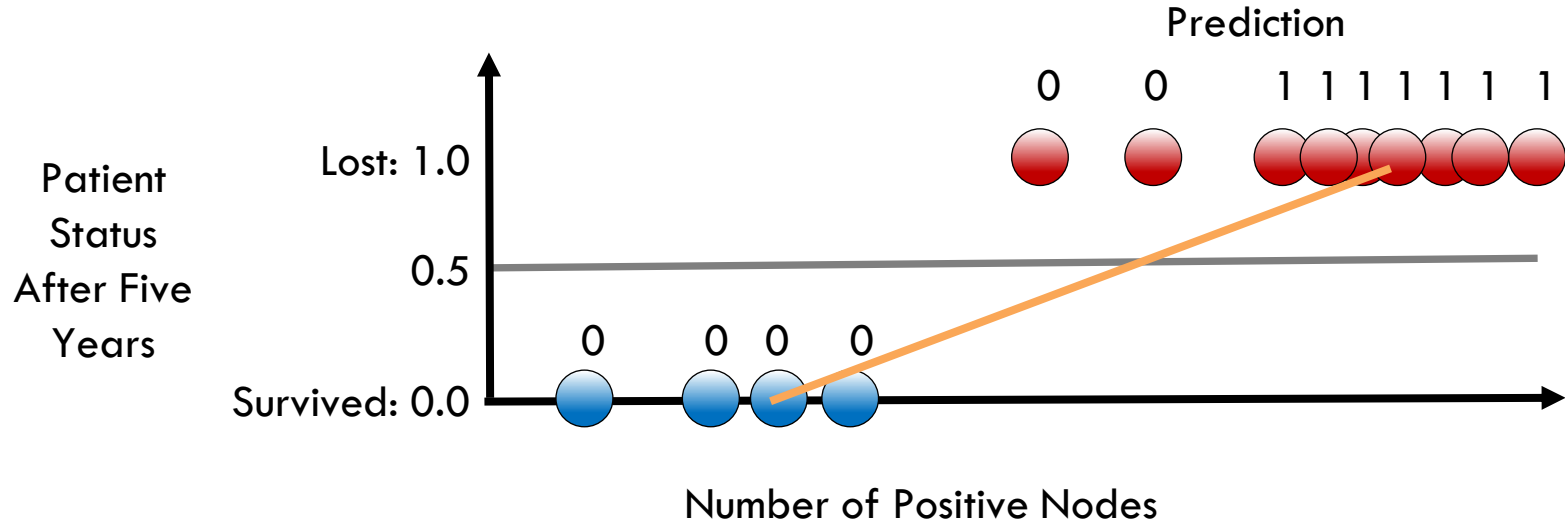
Linear Regression for Classification?



If model result > 0.5 : predict lost

If model result < 0.5 : predict survived

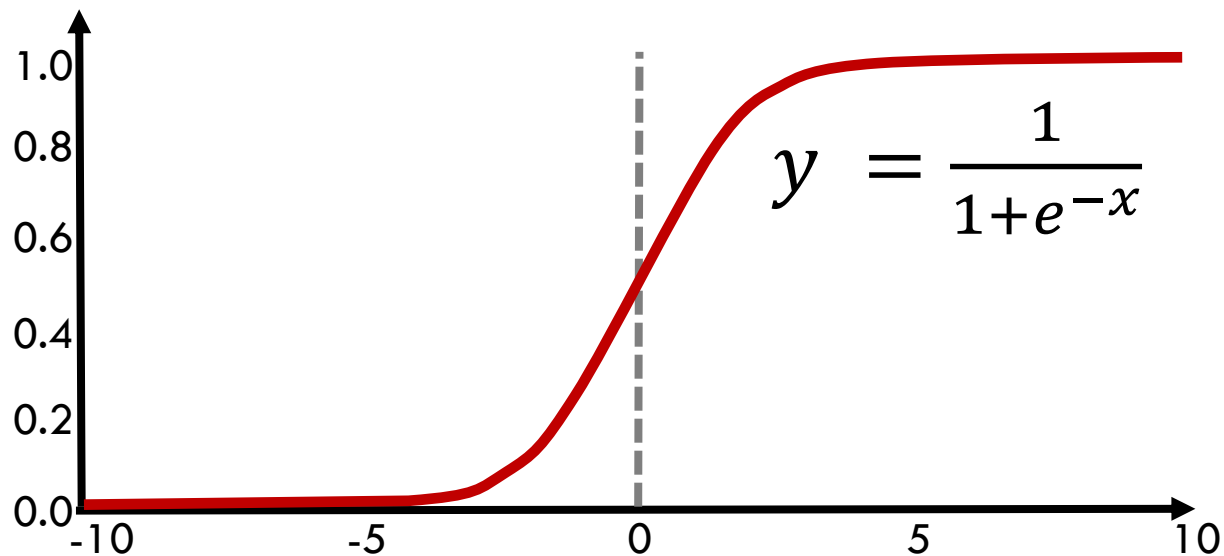
Linear Regression for Classification?



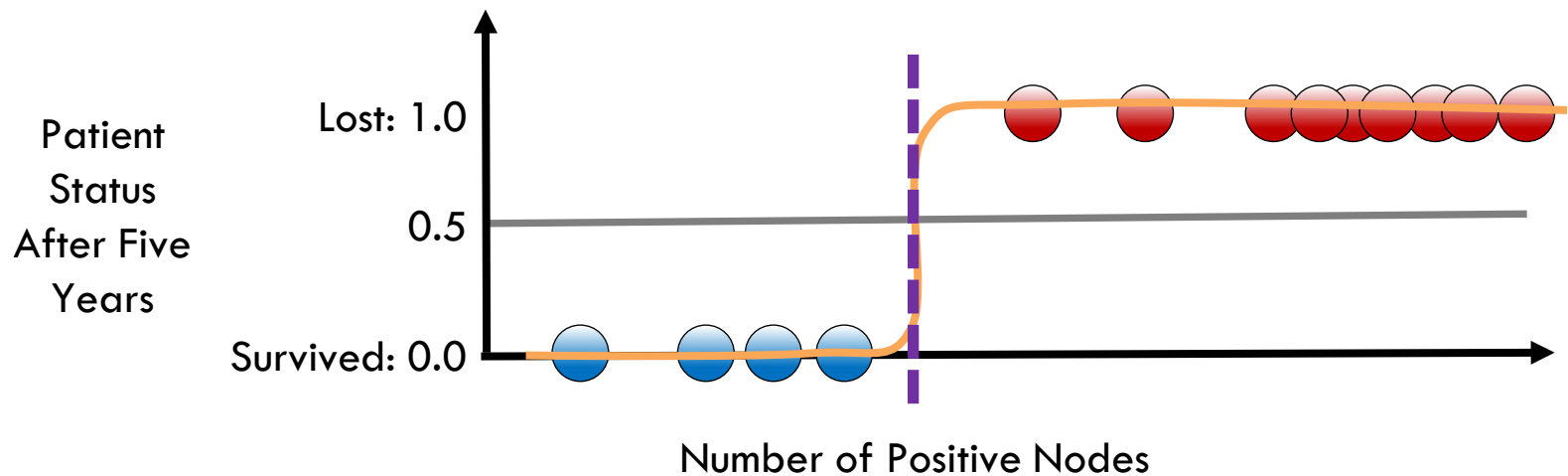
If model result > 0.5 : predict lost

If model result < 0.5 : predict survived

What is this Function?

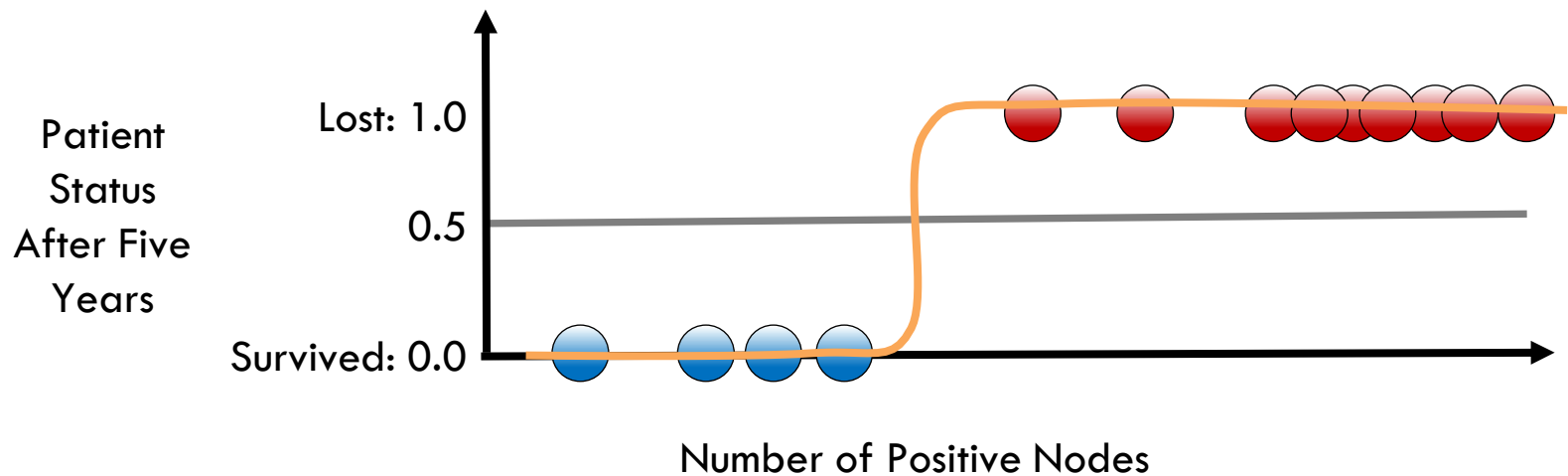


The Decision Boundary



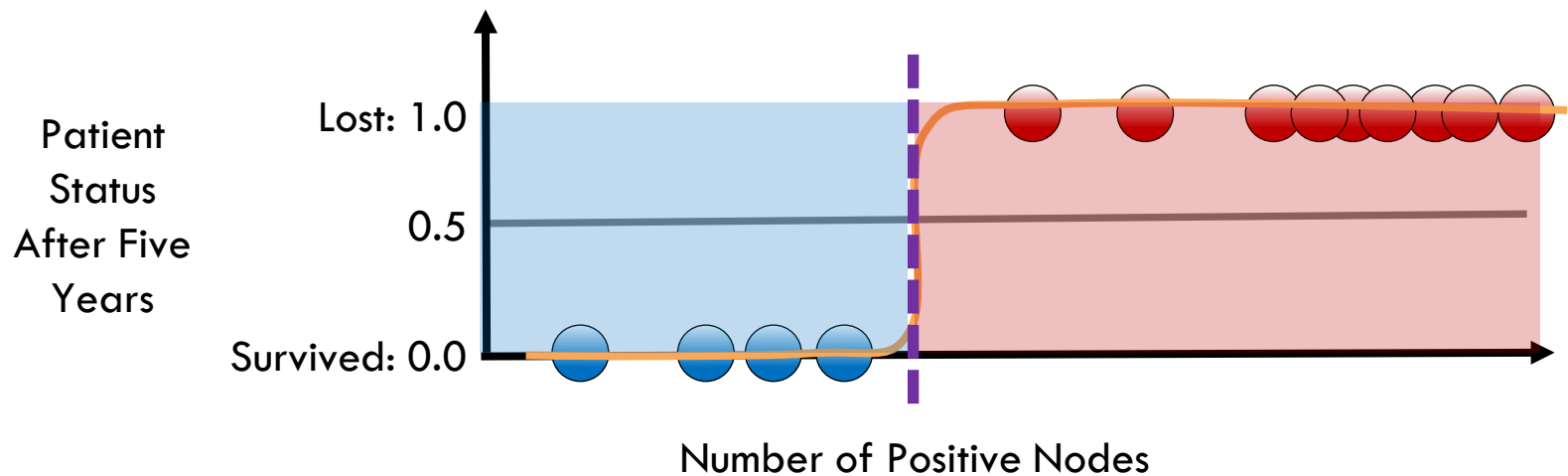
$$y_{\beta}(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

Logistic Regression



$$y_{\beta}(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

The Decision Boundary



$$y_{\beta}(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

Relationship of Logistic to Linear Regression

Logistic
Function

$$P(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

Relationship of Logistic to Linear Regression

Logistic
Function

$$P(x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$

Relationship of Logistic to Linear Regression

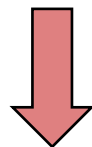
Logistic
Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$

Relationship of Logistic to Linear Regression

Logistic
Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$



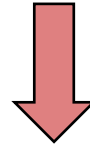
Odds
Ratio

$$\frac{P(x)}{1 - P(x)} = e^{(\beta_0 + \beta_1 x)}$$

Relationship of Logistic to Linear Regression

Logistic
Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$



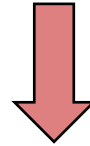
Log
Odds

$$\log \left[\frac{P(x)}{1 - P(x)} \right] = \beta_0 + \beta_1 x$$

Relationship of Logistic to Linear Regression

Logistic
Function

$$P(x) = \frac{e^{(\beta_0 + \beta_1 x)}}{1 + e^{(\beta_0 + \beta_1 x)}}$$



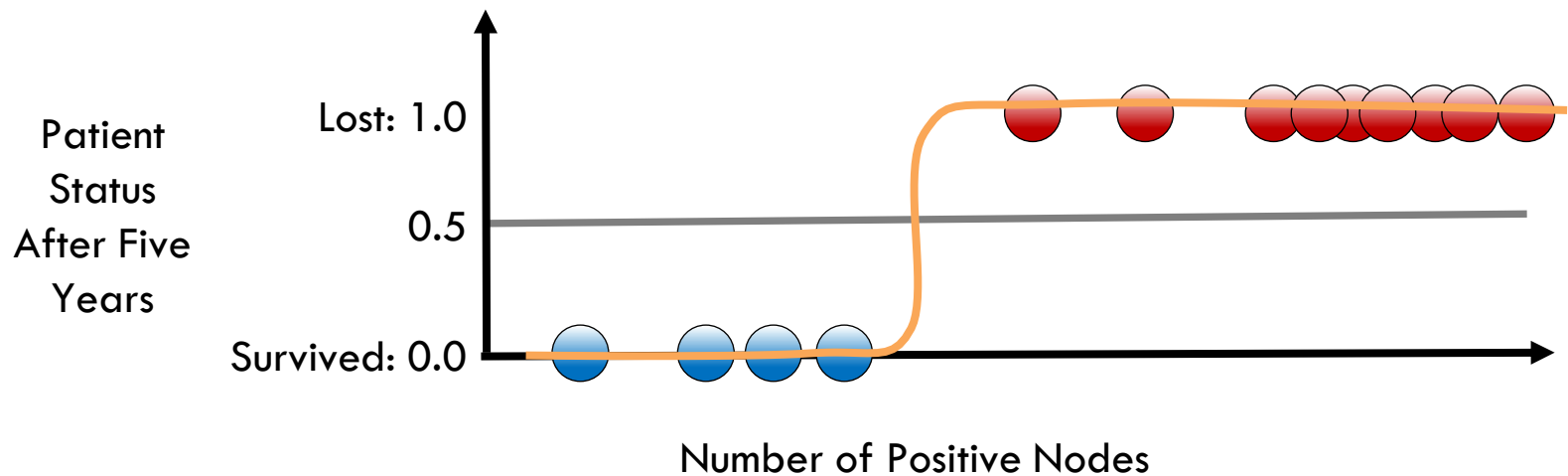
Log
Odds

$$\log \left[\frac{P(x)}{1 - P(x)} \right] = \beta_0 + \beta_1 x$$

Classification with Logistic Regression

One feature (nodes)

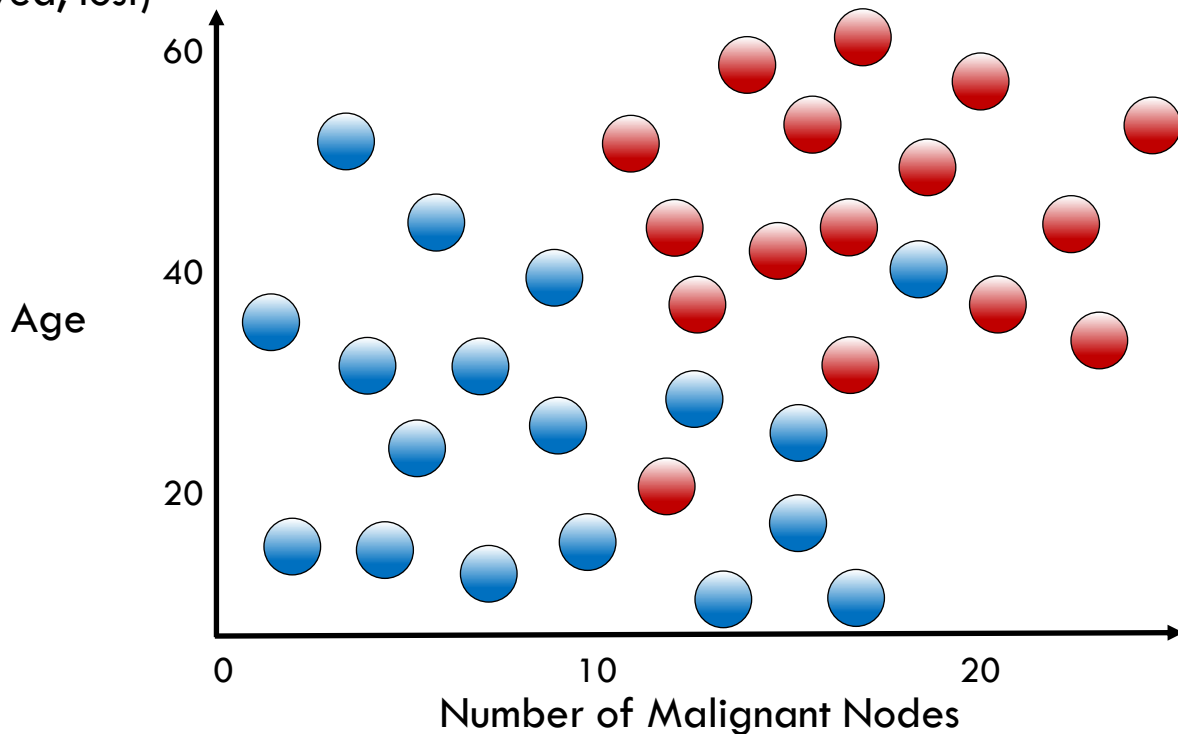
Two labels (survived, lost)



Classification with Logistic Regression

Two features (nodes, age)

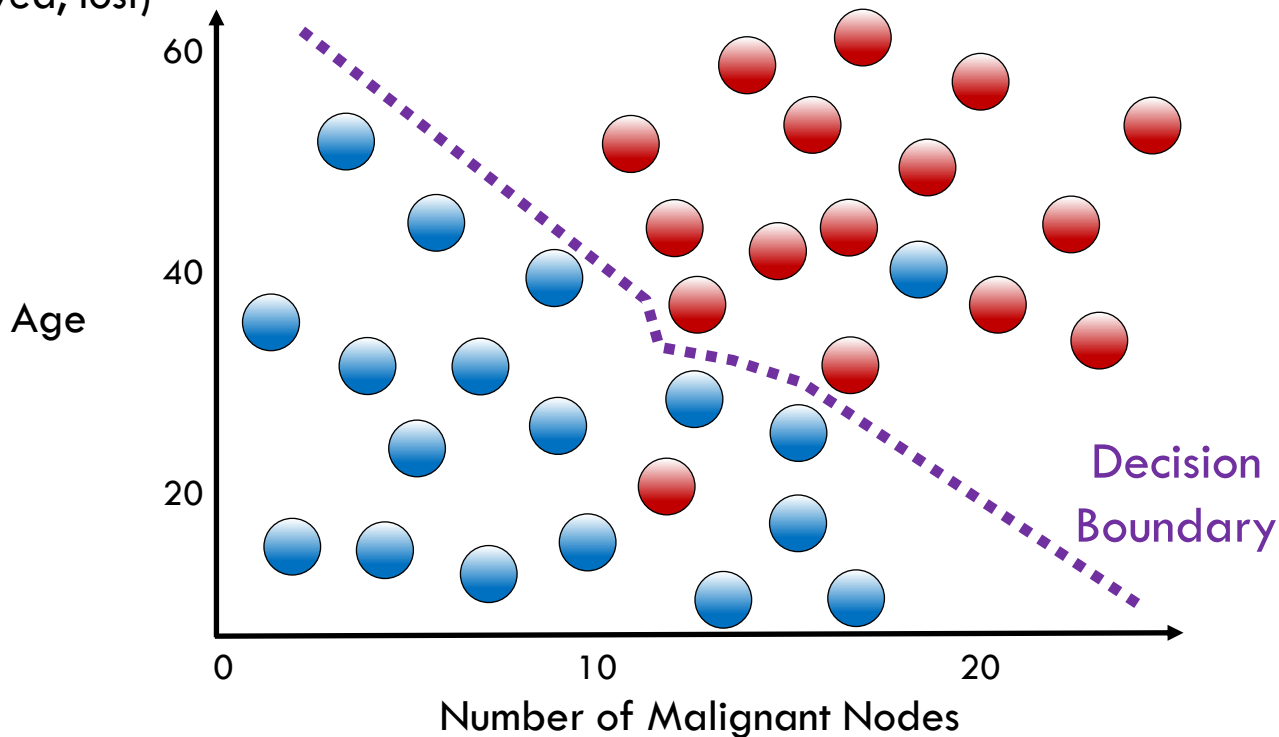
Two labels (survived, lost)



Classification with Logistic Regression

Two features (nodes, age)

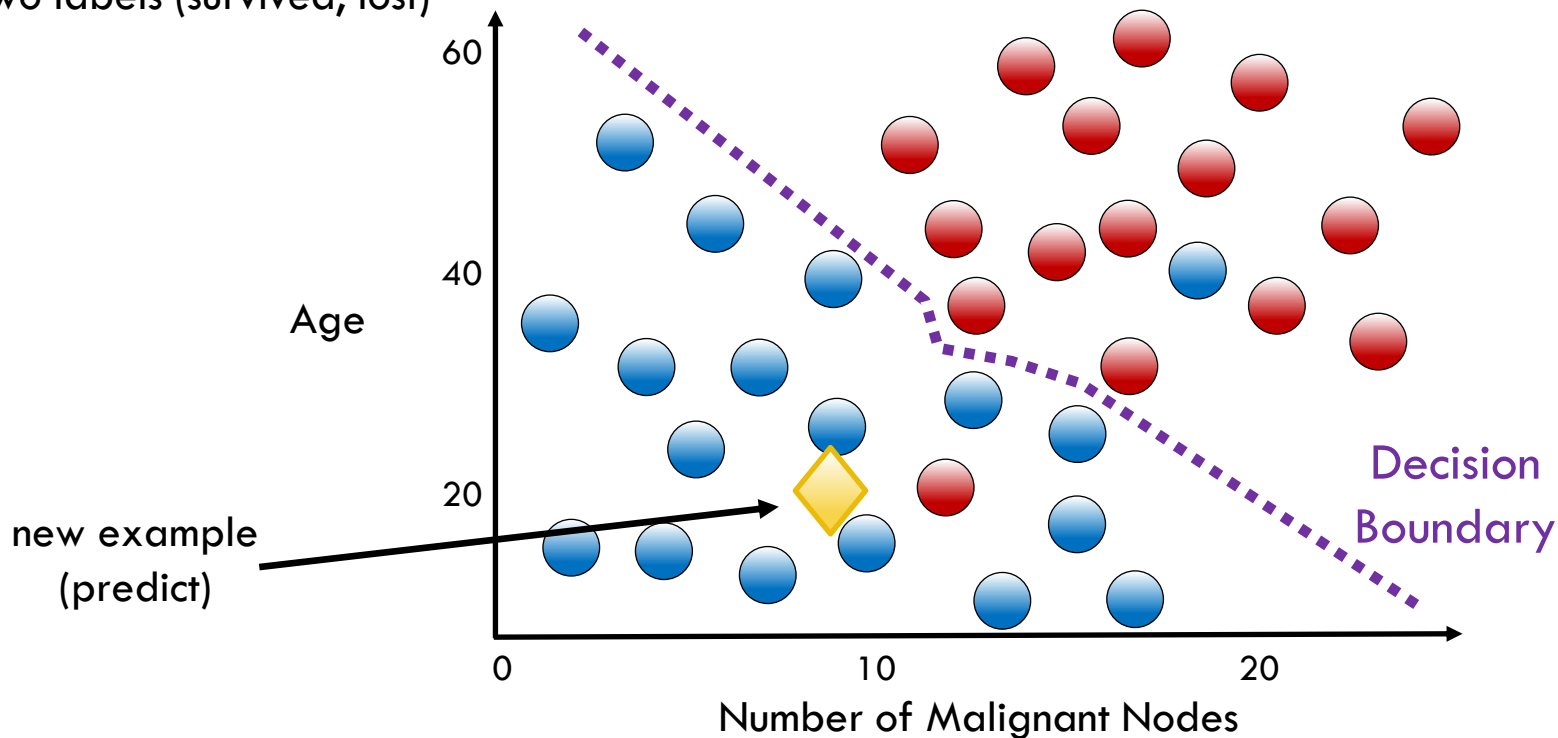
Two labels (survived, lost)



Classification with Logistic Regression

Two features (nodes, age)

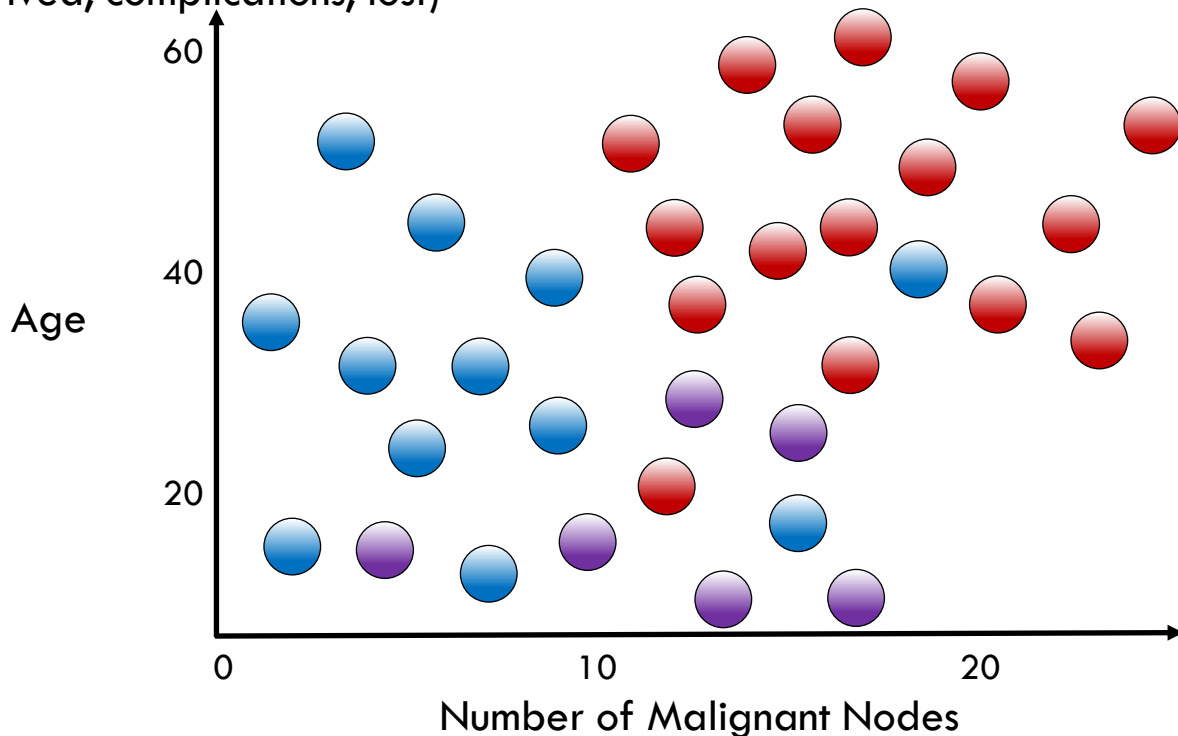
Two labels (survived, lost)



Multiclass Classification with Logistic Regression

Two features (nodes, age)

Three labels (survived, complications, lost)



Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```

Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```



regularization
parameters

Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```

Fit the instance on the data and then predict the expected value

```
LR = LR.fit(X_train, y_train)
```

```
y_predict = LR.predict(X_test)
```


Logistic Regression: The Syntax

Import the class containing the classification method

```
from sklearn.linear_model import LogisticRegression
```

Create an instance of the class

```
LR = LogisticRegression(penalty='l2', c=10.0)
```

Fit the instance on the data and then predict the expected value

```
LR = LR.fit(X_train, y_train)
```

```
y_predict = LR.predict(X_test)
```

Tune regularization parameters with cross-validation: `LogisticRegressionCV`.